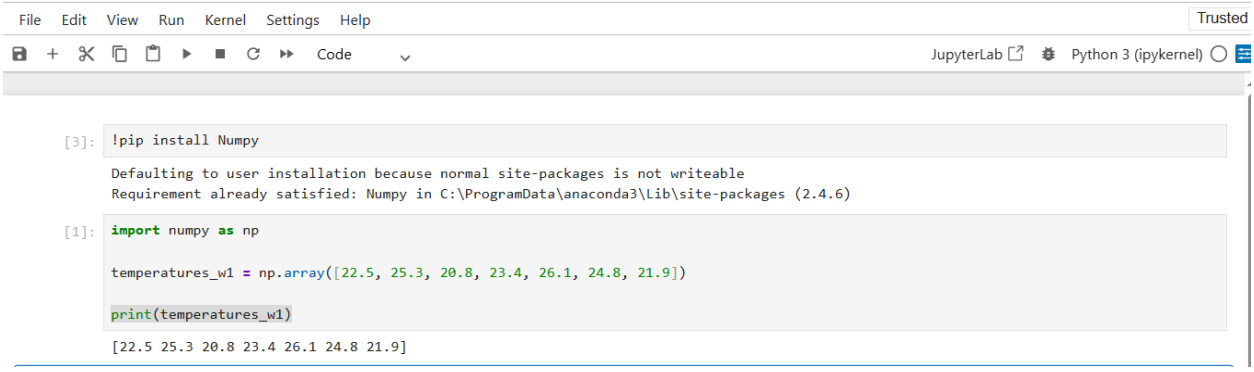
Python DA Assignment 1
Data Analysis using NumPy and Pandas

NAME : UMAYAL.K
SUBMISSION DATE: 26 - 10 - 2026

OperationsNumpy Array:

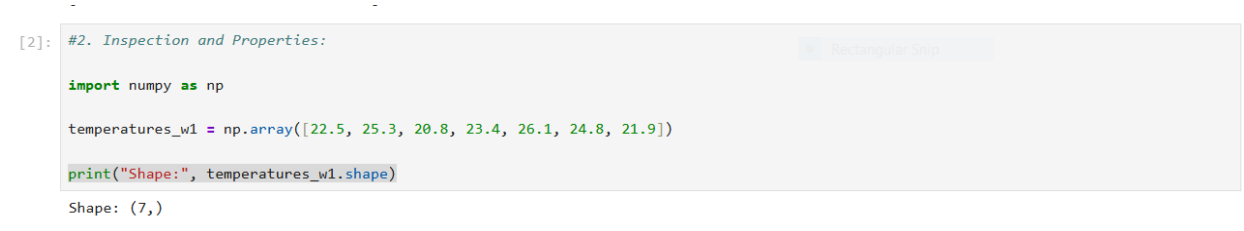
1. Create a 1D NumPy Array:

- This creates a **1-dimensional NumPy array** named `temperatures_w1` containing the Week 1 temperature values.



2. Inspection and Properties:

- `(7,)` means the array has **7 elements** arranged in **one dimension**.
- The comma inside the parentheses indicates that it is a **1D array**.



3. Array Operations:

3(1).Convert Celsius to Fahrenheit

Formula: Fahrenheit = (Celsius * 9/5) + 32

Fahrenheit =(22.5 * 9/5) +32 => 72.5

```
#3. Array Operations:

import numpy as np

temperatures_w1 = np.array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])

# Convert to Fahrenheit
temperatures_f = (temperatures_w1 * 9/5) + 32

print("Temperatures in Fahrenheit:")
print(temperatures_f)
```

```
Temperatures in Fahrenheit:
[72.5  77.54 69.44 74.12 78.98 76.64 71.42]
```

3(2).The maximum, minimum, and mean temperatures for the week:

- **Maximum Temperature:** 26.1 °C
- **Minimum Temperature:** 20.8 °C
- **Mean Temperature:** 23.54 °C (rounded to 2 decimal places)

```
#Find the maximum, minimum, and mean temperatures for the week.

max_temp = np.max(temperatures_w1)
min_temp = np.min(temperatures_w1)
mean_temp = np.mean(temperatures_w1)

print("Maximum Temperature:", max_temp)
print("Minimum Temperature:", min_temp)
print("Mean Temperature:", mean_temp)
```

```
Maximum Temperature: 26.1
Minimum Temperature: 20.8
Mean Temperature: 23.542857142857144
```

4. Array Slicing and Indexing:

- **Extract the temperatures for the first three days of the week.**

```
import numpy as np

temperatures_w1 = np.array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])

#Extract the temperatures for the first three days of the week.

first_three_days = temperatures_w1[:3]

print("First three days:", first_three_days)
```

```
First three days: [22.5 25.3 20.8]
```

- **Extract the temperatures for the weekend (last two days).**

```
#Extract the temperatures for the weekend (last two days).

weekend_temps = temperatures_w1[-2:]

print("Weekend temperatures:", weekend_temps)
```

```
First three days: [22.5 25.3 20.8]
Weekend temperatures: [24.8 21.9]
```

- **Extract the temperatures for the middle three days of the week.**

```
#Extract the temperatures for the middle three days of the week.

middle_three_days = temperatures_w1[2:5]

print("Middle three days:", middle_three_days)
```

```
First three days: [22.5 25.3 20.8]
Weekend temperatures: [24.8 21.9]
Middle three days: [20.8 23.4 26.1]
```

- **These slices use NumPy indexing where the start index is included and the end index is excluded.**

5. Create a 2D Array:

Create a 2D NumPy array named temperatures

```
import numpy as np

temperatures = np.array([
    [22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9],  # Week 1
    [19.2, 22.5, 21.3, 24.0, 23.5, 22.8, 20.1]   # Week 2
])

print(temperatures)
```

```
[[22.5 25.3 20.8 23.4 26.1 24.8 21.9]
 [19.2 22.5 21.3 24.  23.5 22.8 20.1]]
```

- **2 rows** → Week 1 and Week 2
- **7 columns** → 7 days in each week
- Shape **(2, 7)** means **2 weeks × 7 days** of temperature data.

6. Inspect and Slice the 2D Array:

6(1).Inspect and print the shape, data type, and total number of elements in the

temperatures array.

```
#6. Inspect and Slice the 2D Array:
#Inspect and print the shape, data type, and total number of elements in the
#temperatures array.
print("Shape:", temperatures.shape)
print("Data Type:", temperatures.dtype)
print("Total Number of Elements:", temperatures.size)
```

```
Shape: (2, 7)
Data Type: float64
Total Number of Elements: 14
```

`(2, 7)` → 2 weeks and 7 days per week

`float64` → decimal values are stored as floating-point numbers

`14` → total elements = 2 × 7

6(2.a).Extract the temperatures for each week

Week1:

```
#Extract the temperatures for each week
#Week1

week1 = temperatures[0]

print("Week 1 Temperatures:", week1)
```

```
Week 1 Temperatures: [22.5 25.3 20.8 23.4 26.1 24.8 21.9]
```

Week2:

```
#Week2

week2 = temperatures[1]

print("Week 2 Temperatures:", week2)
```

```
Week 1 Temperatures: [22.5 25.3 20.8 23.4 26.1 24.8 21.9]
Week 2 Temperatures: [19.2 22.5 21.3 24.  23.5 22.8 20.1]
```

6(2.b).The weekends (last two days) for both weeks.

```
#The weekends (last two days) for both weeks.

weekend_temps = temperatures[:, -2:]

print("Weekend Temperatures for Both Weeks:")
print(weekend_temps)
```

```
Weekend Temperatures for Both Weeks:
[[24.8 21.9]
 [22.8 20.1]]
```

- **Week 1:** `temperatures[0]`
- **Week 2:** `temperatures[1]`
- **Weekend temperatures for both weeks:** `temperatures[:, -2:]`
- **Array shape:** `(2, 7)`
- **Total elements:** `14`

Pandas Series:

1.Creating Pandas Series:

● Create a Pandas Series named marks with the following data:

```python
#1. Creating Pandas Series:
import pandas as pd

marks = pd.Series(
    [95, 92, 89, 85, 80],
    index=['Rank1', 'Rank2', 'Rank3', 'Rank4', 'Rank5']
)

print(marks)
```

```
Rank1    95
Rank2    92
Rank3    89
Rank4    85
Rank5    80
dtype: int64
```

Activate Windows

2. Indexing and Slicing:

2(1).The mark of the 1st rank student.

2(2).Use the loc accessor the top 3 ranks by specifying their index labels.

2(3).Use the iloc accessor to retrieve the mark of the 3rd rank student.

2(4).Apply a boolean mask to filter the Series and retrieve the ranks where the marks are greater than 90.

- `loc[]` → label-based access
- `iloc[]` → position-based access

```python
import pandas as pd

marks = pd.Series(
    [95, 92, 89, 85, 80],
    index=['Rank1', 'Rank2', 'Rank3', 'Rank4', 'Rank5']
)

# 1. First rank student (position-based)
print("1st Rank Mark:", marks.iloc[0])

# 2. Top 3 ranks using loc
print("\nTop 3 Ranks:")
print(marks.loc[['Rank1', 'Rank2', 'Rank3']])

# 3. Third rank student using iloc
print("\n3rd Rank Mark:", marks.iloc[2])

# 4. Marks greater than 90
print("\nMarks > 90:")
print(marks[marks > 90])
```

Activate Windows
Go to Settings to activate Windows.

```
1st Rank Mark: 95

Top 3 Ranks:
Rank1    95
Rank2    92
Rank3    89
dtype: int64

3rd Rank Mark: 89

Marks > 90:
Rank1    95
Rank2    92
dtype: int64
```

Activate Windows

3. Manipulating Series:

3(1).Modify the mark of the student with the 1st rank to 100.

- `marks['Rank1']` accesses the value associated with the label `'Rank1'`.
- Assigning `100` replaces the old value (`95`).

```python
#3. Manipulating Series:

import pandas as pd

marks = pd.Series(
    [95, 92, 89, 85, 80],
    index=['Rank1', 'Rank2', 'Rank3', 'Rank4', 'Rank5']
)

print("Original Series:")
print(marks)

#1. Modify the mark of the student with the 1st rank to 100

marks['Rank1'] = 100

print("After updating Rank1:")
print(marks)
```

Activate Windows

```
Original Series:
Rank1    95
Rank2    92
Rank3    89
Rank4    85
Rank5    80
dtype: int64
After updating Rank1:
Rank1    100
Rank2     92
Rank3     89
Rank4     85
Rank5     80
dtype: int64
```

3(2).Remove the entry corresponding to the last rank from the Series.

- `drop()` removes the specified index label.
- `'Rank5'` and its value (`80`) are deleted.
- Since `drop()` returns a new Series, we assign it back to `marks`.

```python
#2. Remove the entry corresponding to the last rank

marks = marks.drop('Rank5')

print("After removing Rank5:")
print(marks)
```

```
After removing Rank5:
Rank1    100
Rank2     92
Rank3     89
Rank4     85
dtype: int64
```

3(3).Compute the CGPA by dividing each mark in the Series by 10.

- Pandas performs **vectorized operations**, meaning the calculation is applied to every element automatically.
- 100 / 10 = 10.0
- 92 / 10 = 9.2
- 89 / 10 = 8.9
- 85 / 10 = 8.5

output:

- Rank1 10.0
- Rank2 9.2
- Rank3 8.9
- Rank4 8.5
- dtype: float64

- Notice the data type changes from `int64` to `float64` because division produces decimal values.

```
#3. Compute the CGPA by dividing each mark by 10

cgpa = marks / 10

print("CGPA:")
print(cgpa)
```

```
CGPA:
Rank1    10.0
Rank2     9.2
Rank3     8.9
Rank4     8.5
dtype: float64
```

- `series['label'] = value` → Modify a value.
- `series.drop('label')` → Remove an entry.
- `series / number` → Perform arithmetic on all values in the Series at once.

Pandas DataFrame:

1. Creating Pandas DataFrame:

Create a Pandas DataFrame named transactions with the following data:

- **10 rows** represent 10 transactions.
- **4 columns** represent TransactionID, ProductCategory, Region, and Amount.

```
#Pandas DataFrame:
#1. Creating Pandas DataFrame:
# Create a Pandas DataFrame named transactions with the following data:

import pandas as pd

transactions = pd.DataFrame({
    'TransactionID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],
    'ProductCategory': [
        'Electronics', 'Clothing', 'Electronics', 'Furniture', 'Clothing',
        'Electronics', 'Furniture', 'Clothing', 'Furniture', 'Electronics'
    ],
    'Region': [
        'North', 'South', 'North', 'East', 'West',
        'North', 'East', 'West', 'South', 'North'
    ],
    'Amount': [200, 150, 300, 450, 200, 250, 300, 180, 350, 400]
})

print(transactions)
```

Activate Windows
Go to Settings to activate Windows.

```
   TransactionID ProductCategory Region  Amount
0            101     Electronics  North     200
1            102        Clothing  South     150
2            103     Electronics  North     300
3            104       Furniture   East     450
4            105        Clothing   West     200
5            106     Electronics  North     250
6            107       Furniture   East     300
7            108        Clothing   West     180
8            109       Furniture  South     350
9            110     Electronics  North     400
```

2. Data Exploration:

2(1).Display the transactions DataFrame and its basic information, including head, tail, shape, column names, and data types.

2(1.a).Head (first 5 rows):

```
#2. Data Exploration:
#2(1).Display the transactions DataFrame and its basic information, including
#head, tail, shape, column names, and data types.
#2(1.a).Head (first 5 rows)

print(transactions.head())
```

```
   TransactionID ProductCategory Region  Amount
0            101     Electronics  North     200
1            102        Clothing  South     150
2            103     Electronics  North     300
3            104       Furniture   East     450
4            105        Clothing   West     200
```

2(1.b).Tail (last 5 rows)

```
#2(1.b).Tail (last 5 rows)

print(transactions.tail())
```

```
   TransactionID ProductCategory Region  Amount
5            106     Electronics  North     250
6            107       Furniture   East     300
7            108        Clothing   West     180
8            109       Furniture  South     350
9            110     Electronics  North     400
```

2(1.c).Shape

```
#2(1.c).Shape

print("Shape:", transactions.shape)
```

```
Shape: (10, 4)
```

2(1.d).Column Names

```
#2(1.d).Column Names

print("Columns:", transactions.columns.tolist())
```

```
Columns: ['TransactionID', 'ProductCategory', 'Region', 'Amount']
```

2(1.e).Data Types

```
#2(1.e).Data Types

print(transactions.dtypes)
```

```
TransactionID      int64
ProductCategory      str
Region               str
Amount             int64
dtype: object
```

Complete Information

```
#Complete Information

transactions.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   TransactionID    10 non-null     int64
 1   ProductCategory  10 non-null     str
 2   Region           10 non-null     str
 3   Amount           10 non-null     int64
dtypes: int64(2), str(2)
memory usage: 593.0 bytes
```

2(2).Display only the 'ProductCategory' and 'Amount' columns.

```
#2(2).Display only the 'ProductCategory' and 'Amount' columns.

print(transactions[['ProductCategory', 'Amount']])
```

```
  ProductCategory  Amount
0     Electronics     200
1        Clothing     150
2     Electronics     300
3       Furniture     450
4        Clothing     200
5     Electronics     250
6       Furniture     300
7        Clothing     180
8       Furniture     350
9     Electronics     400
```

2(3).Retrieve the last 3 columns of the DataFrame using iloc or loc.

```
#2(3).Retrieve the last 3 columns of the DataFrame using iloc or loc.

print(transactions.iloc[:, -3:])
```

```
  ProductCategory Region  Amount
0     Electronics  North     200
1        Clothing  South     150
2     Electronics  North     300
3       Furniture   East     450
4        Clothing   West     200
5     Electronics  North     250
6       Furniture   East     300
7        Clothing   West     180
8       Furniture  South     350
9     Electronics  North     400
```

2(4).Filter rows where the 'Region' is 'North' and 'Amount' is greater than 200.

```
#2(4).Filter rows where the 'Region' is 'North' and 'Amount' is greater than 200.

filtered_data = transactions[
    (transactions['Region'] == 'North') &
    (transactions['Amount'] > 200)
]

print(filtered_data)
```

```
   TransactionID ProductCategory Region  Amount
2            103     Electronics  North     300
5            106     Electronics  North     250
9            110     Electronics  North     400
```

2(5).Find the value counts for the 'ProductCategory' column.

```
#2(5).Find the value counts for the 'ProductCategory' column.

print(transactions['ProductCategory'].value_counts())
```

```
ProductCategory
Electronics    4
Clothing       3
Furniture      3
Name: count, dtype: int64
```

2(6).Find the unique values in the 'Region' column.

```
#2(6).Find the unique values in the 'Region' column.

print(transactions['Region'].unique())
```

```
<ArrowStringArray>
['North', 'South', 'East', 'West']
Length: 4, dtype: str
```

2(7).Group by 'Region' and find the mean amount for each region.

- **East:** (450 + 300) / 2 = **375.0**
- **North:** (200 + 300 + 250 + 400) / 4 = **287.5**
- **South:** (150 + 350) / 2 = **250.0**
- **West:** (200 + 180) / 2 = **190.0**

```
#2(7).Group by 'Region' and find the mean amount for each region.

mean_amount = transactions.groupby('Region')['Amount'].mean()

print(mean_amount)
```

```
Region
East     375.0
North    287.5
South    250.0
West     190.0
Name: Amount, dtype: float64
```

3. Manipulating the DataFrame:

3(1).Modify the 'Amount' for the transaction with TransactionID 102 to 165:

- `transactions['TransactionID'] == 102` finds the row with TransactionID 102.
- `loc` selects that row and the `Amount` column.
- `165` replaces the old value `150`.

```
#3. Manipulating the DataFrame:
#1. Modify the Amount for TransactionID 102 to 165

transactions.loc[transactions['TransactionID'] == 102, 'Amount'] = 165

print(transactions)
```

```
   TransactionID ProductCategory Region  Amount
0            101     Electronics  North     200
1            102        Clothing  South     165
2            103     Electronics  North     300
3            104       Furniture   East     450
4            105        Clothing   West     200
5            106     Electronics  North     250
6            107       Furniture   East     300
7            108        Clothing   West     180
8            109       Furniture  South     350
9            110     Electronics  North     400
```

Activate Windows

3(2).Add a new column 'Discount' by calculating 10% of the 'Amount'.

- Discount = Amount × 10%
- 200 × 0.10 = 20
- 165 × 0.10 = 16.5
- 300 × 0.10 = 30

```
#3(2).Add a new column 'Discount' by calculating 10% of the 'Amount'.

transactions['Discount'] = transactions['Amount'] * 0.10

print(transactions)
```

```
   TransactionID ProductCategory Region  Amount  Discount
0            101     Electronics  North     200      20.0
1            102        Clothing  South     165      16.5
2            103     Electronics  North     300      30.0
3            104       Furniture   East     450      45.0
4            105        Clothing   West     200      20.0
5            106     Electronics  North     250      25.0
6            107       Furniture   East     300      30.0
7            108        Clothing   West     180      18.0
8            109       Furniture  South     350      35.0
9            110     Electronics  North     400      40.0
```

3(3). Remove the row with TransactionID 109:

- This keeps all rows **except** the one where TransactionID is 109.
- Removed row : 109 Furniture South 350

```
#3(3). Remove the row with TransactionID 109

transactions = transactions[transactions['TransactionID'] != 109]

print(transactions)
```

```
   TransactionID ProductCategory Region  Amount  Discount
0            101     Electronics  North     200      20.0
1            102        Clothing  South     165      16.5
2            103     Electronics  North     300      30.0
3            104       Furniture   East     450      45.0
4            105        Clothing   West     200      20.0
5            106     Electronics  North     250      25.0
6            107       Furniture   East     300      30.0
7            108        Clothing   West     180      18.0
9            110     Electronics  North     400      40.0
```

3(4). Delete the 'Discount' column:

- `drop()` removes a row or column.

- `axis=1` means column.
- `inplace=True` updates the DataFrame directly.

Key Methods Used:

- `loc[]` → Update specific values.
- `DataFrame['new_column'] = ...` → Add a new column.
- Boolean filtering (`!=`) → Remove specific rows.
- `drop(..., axis=1)` → Delete a column.

```
#3(4). Delete the 'Discount' column

transactions.drop('Discount', axis=1, inplace=True)

print(transactions)
```

```
   TransactionID ProductCategory Region  Amount
0            101     Electronics  North     200
1            102        Clothing  South     165
2            103     Electronics  North     300
3            104       Furniture   East     450
4            105        Clothing   West     200
5            106     Electronics  North     250
6            107       Furniture   East     300
7            108        Clothing   West     180
9            110     Electronics  North     400
```